BulSUScholar: Three-Revision Implementation Audit

Source inspection date: 2 November 2026. Investigation and documentation only. Application code, manuscript, database and deployment configuration are not changed. Scope follows Task Extract the Figures.txt: Revisions 1-3; Revision 4 development remains on hold.

SECTION 1 - System Audit Summary

The repository contains a React/Vite frontend (src), FastAPI Python services (backend), Supabase PostgreSQL migrations and private storage references (supabase), shared location catalogs (shared), and thesis/test documentation (docs). Evidence was traced across routes, frontend service calls, backend validation and SQL transaction functions. Relevant manuscript files are the DOCX and PDF named CENTRALIZED-SCHOLARSHIP-MANAGEMENT-AND-TRACKING-SYSTEM-FOR-BULACAN-STATE-UNIVERSITY. The DOCX is the editing authority; the PDF supplies stable current page references.

Status legend: IMPLEMENTED = traced in current source, not proof of deployment or effectiveness. RECORDED TEST EVIDENCE = existing execution artifacts inspected, not newly rerun. PARTIAL = source exists but has a documented limitation. NOT VERIFIED = no current runtime/configuration evidence. No authenticated screenshots, production operations, uploads or migrations were performed for this audit.

Recorded evidence: docs/ThesisPaper/Algorithm_Metrics_Audit (28 October 2026) contains 48 passing matching/roster regression tests, 33 passing document tests, seven isolated SQL assertions, ten synthetic matching pairs and three synthetic TEST PDFs. These are bounded local results, not population-level student accuracy, OCR accuracy, security certification or proof of live deployment. Prior inspection matched 285 source-file hashes to the recorded snapshot; no fresh test run is claimed.

Ten-pair synthetic results: TP=5, FP=1, FN=0, TN=4; precision=83.33%, recall=100%, derived F1=90.91%, accuracy=90%. Document evidence reports COR 16/16 and ROG 9/9 selected fields on the small fixture set. Four negative-validation gaps were reproduced in isolated real-parser tests; two OCR probes were dependency-blocked. These limitations must appear beside any test claims.

Manuscript locations: specific objectives printed pp. 5-6 (PDF viewer pp. 8-9); Chapter IV begins printed p. 125 (viewer p. 128). In the inspected relevant body pages, viewer numbering is printed numbering plus three. Direct-body paragraph indices in the machine-readable anchor file are zero-based and exclude table paragraphs.

SECTION 2 - Revision 1 Findings

Verified method and inputs

The implemented method is Weighted Recommendation Scoring: rule-based candidate filtering followed by deterministic weighted ranking. It is not a trained machine-learning Decision Tree. A decision-flow diagram may represent its branches, but must not describe learning, model fitting, training data or predicted approval probabilities. Entry: POST /scholarships/recommend; backend/main.py recommend_scholarships_endpoint loads the authorized stored student and calls backend/scholarship_rules.py:137-216.

Student scoring attributes are GWA (gwa/currentGwa/generalWeightedAverage) and flat province, city and barangay. Candidate attributes are applicationsBlocked, applicationEnabled, applyOpen, minimumGwa/minGwa/minimumGrade, provider/title names for policy fallback, rosterCount and any of profileImageUrl/authorImageUrl/imageUrl. Course, year, income, birthdate, contact number, document approval and academic-cycle compliance do not contribute to this core score. The endpoint replaces caller-provided student data, but does not rebuild the supplied candidate list authoritatively.

Frontend candidate layer: src/services/recommendedScholarshipService.js loads cached grantors, announcements and roster counts (20-second cache), excludes archived/disabled grantors and non-open/hidden/archived announcements, and builds enabled, unblocked announcement candidates. Roster counts skip archived rows but are not a strict current-cycle active-award count. Application requirements and slot metadata may be carried on candidates without being evaluated by the ranker. The page additionally handles commitments, unresolved multiple-award conflicts, invitations and same-grantor cooldowns. Final application/waitlist validation is a separate server transaction.

Step	Decision point	Exact condition	YES path	NO path	Source lines in backend/scholarship_rules.py
C1	High-grade preparation	student_gwa is not None and student_gwa <= 1.75	Popularity weight later = 30	Popularity weight later = 22	142-143
C2	Next candidate?	for item in scholarships	Evaluate C3	Sort and return	146,204-216
C3	Blocked?	item.get("applicationsBlocked") is True	Exclude; next candidate	C4	147-148
C4	Disabled without override?	applicationEnabled is False and applyOpen is not True	Exclude; next candidate	C5	149-150
C5	Minimum missing?	_to_float(minimumGwa or minGwa or minimumGrade) is None	Use provider policy; fallback 2.25	Keep parsed minimum	152-155
C6	GWA missing or too high?	student_gwa is None or student_gwa > minimum_gwa	Exclude; next candidate	Score candidate	156-161
C7	Strong grade margin?	max(0, minimum_gwa - student_gwa) >= 0.5	Strong GWA reason	Meets minimum GWA reason	160-165
C8	Positive max roster?	max_roster > 0	Compute relative popularity	Popularity = 0	167-169
C9	Positive candidate roster?	roster_count > 0	Add roster reason	No roster reason	170-171
C10	Nearest reason?	location_score >= 22	Add nearest reason	C11	173-175
C11	Same-area reason?	location_score >= 12	Add same-province reason	C12	176-177
C12	Pampanga to Bulacan?	normalized student province == pampanga and candidate province == bulacan	Add reduced-priority reason	No location reason	178-179

Step	Decision point	Exact condition	YES path	NO path	Source lines in backend/scholarship_rules.py
C13	Candidate image exists?	profileImageUrl or authorImageUrl or imageUrl	Completeness = 10	Completeness = 8	181-183
C14	Top label?	raw score >= 78	Best Scholarship Match For You	Check label reason precedence	185-186;265-274
C15	Label alternatives	Nearest reason, else roster reason, else Strong GWA reason	First applicable near/popular/strong label	Available Scholarship Match	265-274

Execution nuance: C1 is the first preparatory conditional expression; C3 is the first scholarship-exclusion branch. max_roster is computed over all input candidates before exclusion. C5 uses Python or expressions, so a false-like value may fall through to the next alias. applyOpen=True overrides only C4; applyOpen=False alone does not necessarily exclude an otherwise enabled candidate. _to_float returns None for unparseable values but contains no explicit finite-number or valid 1-5 GWA-range guard.

Factor and label rules

Factor	Actual calculation / branches	Interpretation
Grade	margin=max(0, minimumGwa-studentGwa); grade=min(40,24+12*margin)	Lower numeric GWA is better; base 24 after passing the GWA filter.
Popularity	rosterCount/maxRoster * (30 if GWA<=1.75 else 22), or 0 if maxRoster<=0	Relative to input roster maximum, including subsequently excluded candidates.
Location	All six address fields empty: 8. Otherwise province exact +10; else Pampanga-to-Bulacan +4; else nonempty grantor province +2. Independent city equality +10 and barangay equality +5; cap25.	Normalization is lowercase alphanumeric text. No distance calculation; city equality does not require province equality.
Completeness	8 base; +2 for any candidate image	Not student-profile completeness or financial need.
Total and labels	Sum four factors. Raw score>=78 gets top label; else nearest reason, roster reason, strong GWA reason, available.	78 is a label threshold, not an approval or exclusion threshold; score is not a percentage.
Output	Original item, rounded score (4 decimals), label, first four reasons, criteria with GWA/minimum/roster and three factor scores (2 decimals). Sort stored rounded score descending.	Stable ties retain input order in the backend. Nominal maxima for nonnegative inputs: 97 ordinary /105 high-grade.

Fallback/display layer: local ranking is used only for backend_unavailable or request_timeout, with degraded=true. It is not guaranteed identical for arbitrary raw inputs: local minimum fallback is 2.25 and JavaScript numeric parsing differs. The page merges invitations and eligible reapplications (priority1), admin recommendations (priority2), and algorithm results (priority2 when roster-scoped, otherwise3). It deduplicates scholarships and sorts priority ascending, then score descending. Thus the highest algorithm score need not be the first displayed card. No recommendation automatically approves or awards a scholarship.

R1 - Backend recommendation decision flow

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
R1S	Start recommendation	Start/End	none	next: R1I	Backend
R1I	Receive authorized stored student and supplied candidates	Input/Output	R1S	next: R1P	API
R1P	Parse GWA; compute input maximum roster and high-grade flag	Process	R1I	next: R1N	Backend
R1N	Another candidate?	Decision	R1P; R1X; R1L	YES: R1B; NO: R1O	Backend loop
R1B	applicationsBlocked is True?	Decision	R1N [YES]	YES: R1X; NO: R1D	Backend
R1D	Disabled and applyOpen is not True?	Decision	R1B [NO]	YES: R1X; NO: R1M	Backend
R1M	Parsed minimum GWA missing?	Decision	R1D [NO]	YES: R1F; NO: R1G	Backend
R1F	Resolve provider minimum; default 2.25	Process	R1M [YES]	next: R1G	Backend
R1G	Student GWA missing or greater than minimum?	Decision	R1M [NO]; R1F	YES: R1X; NO: R1V	Backend
R1X	Exclude this candidate; advance iterator	Process	R1B [YES]; R1D [YES]; R1G [YES]	next: R1N	Backend
R1V	Calculate factors and ordered reasons using C7-C13	Process	R1G [NO]	next: R1L	Backend
R1L	Apply label precedence C14-C15; append rounded score and criteria	Process	R1V	next: R1N	Backend
R1O	Sort score descending and return recommendations, algorithm and weights	Input/Output	R1N [NO]	next: R1E	API
R1E	End recommendation	Start/End	R1O	end	Backend

R1 is a readable overview. C7-C15 in the decision table are the explicit scoring/label branch expansion for R1V/R1L; do not draw them as additional hard eligibility gates. Draw frontend candidate preparation and final priority ordering as separate surrounding subprocesses, not as conditions executed inside recommend_scholarships.

SECTION 3 - Revision 2 Findings

Use the following independent processes. Each chart begins at its own trigger and ends at its own result; a process reference is not evidence that every workflow invokes every matching algorithm. Solid edges are business flow; dashed edges lead only to algorithm/validation reference boxes. Detailed calculations belong in the reference explanations, not in main process nodes.

Workflow	Trigger	Main process	Algorithms/mechanisms	Result	Source
W1	Registration files and final signup	Parse signup batch, validate profile and availability, finalize pending account	PDF text/layout parser; exact ID/cycle; signup roster-name helper	Pending account or correction; activation is W2	backend/signup_service.py:172-306
W2	Confirmed email / administrator approval	Manual pending review or automatic activation; separate exact roster reconciliation	Required latest signup documents; per-admin preview gate; transactional activation	Active account or pending/conflict; welcome delivery separate	backend/student_lifecycle_service.py; backend/student_account_review_service.py; supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql
W3	Active student replacement / profile submit	Academic/identity automatic review; application-profile manual revision review	Document-specific policy versus profile validation/latest version	Approved replacement or pending profile; correction on failure	backend/student_profile_service.py:885-967
W4	Admin student table load / grantor scholar edit	Group suspected student duplicates or block matching scholar edit	Weighted Record Linkage with Levenshtein Similarity	Canonical display grouping versus edit blocking; not deletion	backend/grantor_algorithms.py:206-262
W5	Existing manual/roster preview and commit	Validate current-cycle imported records; authoritative commit recheck	Exact IDs if both supplied; otherwise complete name/course/year profile key	Blocked batch, idempotent import, linked assignment or unlinked row	backend/roster_workflow_service.py:258-465
W6	Verified session/account reconciliation; full-admin resolution	Try official assignment; record conflict or resolve selected candidate	Exact roster ID/name/cycle and award/commitment guards	Assigned award, unresolved conflict or reasoned resolution	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:69-254,327-403
W7	Student Apply or waitlist action	Check identity, policy, existing conflicts and capacity transactionally	Normalized application/roster/commitment and location eligibility	Application reservation, waitlist or structured blocker	backend/scholarship_choice_service.py; supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:447-582
W8	Request Materials stage and confirm	Cached readiness then same transactional preflight at commit	Normalized current-cycle documents, profile revision, requirements and state	One request plus commitment/competing closure or 409	backend/roster_workflow_service.py; supabase/migrations/20261012100000_repair_normalized_materials_workflow.sql

Document extraction and workflow-specific rules

backend/document_scanner.py parse_pdf_document:926 onward reads PDF text, extracts year from layout, and for ROG tries table Final-grade cells, then positioned words, then text Final-grade fallback. parse_document combines student ID, name parts/full name, course, year, section, GWA, academic year/semester, academic-concern flags and title/type validity metadata. These are extracted fields, not proof that each is checked for equality. It does not implement one universal comparison of birthdate, school, street address and contact number

from COR/ROG.

Signup batch: nonempty files up to 10MiB; allowed identity image/PDF types; COR/ROG PDF processing with optional password assistance. COR must satisfy title, configured COR/advising policy, exact normalized supplied ID and current cycle. Year may be detected or declared, with a mismatch review flag. First-year first-semester ROG is exempt; otherwise ROG title, previous cycle and academic concerns are checked. The batch function does not compare ROG owner ID. Final signup separately checks required profile/contact fields, availability, batch/cycle/year progression and roster identity. Browser ROG identity scoring is advisory in its caller; an inconclusive match is logged rather than a universal hard backend gate.

Active replacement: upload_student_document parses COR/ROG and checks title/policy; identity checks allowed kind/active exception. A successful version is privately stored and system-reviewed before replacing compatibility pointers. This path does not repeat all signup owner-ID, cycle and ROG-concern checks. Student Application Profiles instead validate draft/signature, produce a private PDF revision and remain pending for latest-version administrator review. Identity is not a recurring semester requirement. Historical submissions are retained.

Known gaps from recorded isolated execution: profile COR owner mismatch and profile ROG period mismatch were accepted in the tested path; signup batch accepted missing COR course and mismatched ROG owner ID. Passing parser/title checks must not be described as comprehensive academic-document verification. PDF extraction is not a guarantee of document authenticity or virus scanning.

W1 - Signup academic validation and pending-account creation

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W1S	Start registration	Start/End	none	next: W1I	Student
W1I	Submit profile, COR, applicable ROG and identity	Input/Output	W1S	next: W1P	Frontend/API
W1P	Read and parse private signup file batch	Process	W1I	next: W1D	Backend
W1D	Batch file and signup document rules pass?	Decision	W1P	YES: W1V; NO: W1X	Backend
W1V	Validate required signup data, availability and roster identity	Process	W1D [YES]	next: W1A	Backend
W1A	Final signup validation passes?	Decision	W1V	YES: W1C; NO: W1X	Backend
W1C	Create Auth registration and finalize pending signup records	Process	W1A [YES]	next: W1O	Auth/frontend/backend
W1O	Return pending account; send confirmation; activation continues in W2	Input/Output	W1C	next: W1E	API
W1X	Return specific correction or service failure; do not claim successful signup	Input/Output	W1D [NO]; W1A [NO]	next: W1E	API
W1E	End registration attempt	Start/End	W1O; W1X	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W1P	PDF text/layout and Final-column extraction	backend/document_scanner.py:926 onward	Extract document metadata; no authenticity guarantee
W1V	Exact signup identity and roster-name comparison	backend/signup_service.py:575-612	Separate roster-ID lookup and name-consistency validation

W2 - Email confirmation and account activation

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W2S	Start confirmed-email callback	Start/End	none	next: W2P	Student/Auth
W2P	Verify confirmed identity and promote pending email state	Process	W2S	valid: W2D; invalid/error: W2X	Backend/RPC
W2D	Already active?	Decision	W2P [valid]	YES: W2O; NO: W2M	Backend
W2M	Manual account review enabled?	Decision	W2D [NO]	YES: W2I; NO: W2R	Backend
W2I	Administrator previews required signup documents and submits one approval	Input/Output	W2M [YES]	next: W2V	Full administrator
W2V	Confirmed email, required copies, acceptable reviews and admin preview gate pass?	Decision	W2I	YES: W2A; NO: W2B	Backend
W2R	All applicable required signup documents approved?	Decision	W2M [NO]	YES: W2A; NO: W2B	Backend
W2A	Activate account using approval RPC; attempt roster reconciliation W6	Process	W2V [YES]; W2R [YES]	success: W2W; failure: W2X	Database
W2W	For newly active account only, attempt welcome email; retain activation on mail failure	Process	W2A [success]	next: W2O	Backend
W2B	Keep pending; return missing review or readiness reason	Input/Output	W2V [NO]; W2R [NO]	next: W2E	API
W2X	Return structured activation failure	Input/Output	W2P [invalid/error]; W2A [failure]	next: W2E	API
W2O	Return active/already-active and separate delivery status	Input/Output	W2D [YES]; W2W	next: W2E	API
W2E	End activation attempt	Start/End	W2B; W2X; W2O	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W2V	Latest required document selection and preview audit	backend/student_account_review_service.py:147-223	Preview belongs to this administrator; download is not preview

W3 - Document replacement versus profile review

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W3S	Start document submission	Start/End	none	next: W3D	Student
W3D	Student Application Profile?	Decision	W3S	YES: W3F; NO: W3U	Frontend
W3F	Validate profile and signature; create pending private PDF revision	Process	W3D [YES]	success: W3I; failure: W3X	Backend
W3I	Administrator previews newest profile; chooses approve or reject	Input/Output	W3F [success]	next: W3R	Administrator
W3R	Newest pending version and valid decision/reason?	Decision	W3I	YES: W3V; NO: W3X	Backend
W3V	Record profile decision and review; synchronize application snapshot on approval	Process	W3R [YES]	next: W3O	Backend/database
W3U	Validate owner, type, size and document-specific parser/policy	Process	W3D [NO]	pass: W3A; failure: W3X	Backend
W3A	Store immutable replacement; record system approval; synchronize current pointers	Process	W3U [pass]	success: W3O; failure: W3X	Backend/database
W3O	Return version and its automatic or manual review status	Input/Output	W3V; W3A [success]	next: W3E	API
W3X	Return validation, stale-version or service error	Input/Output	W3F [failure]; W3R [NO]; W3U [failure]; W3A [failure]	next: W3E	API
W3E	End submission/review attempt	Start/End	W3O; W3X	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W3U	Document-specific parser and configured policy	backend/student_profile_service.py:885-967	Do not substitute stronger signup rules
W3R	Latest-version and pending-state guards	backend/student_profile_service.py:999-1063	Avoid stale profile decisions

W4 - Suspected duplicate handling: two independent consumers

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W4S	Start student-list check or scholar edit	Start/End	none	next: W4I	Administrator/grantor
W4I	Supply permitted record set and candidate if editing	Input/Output	W4S	next: W4P	Frontend/API
W4P	Compare normalized records using this consumer matching rule	Process	W4I	next: W4D	Backend
W4D	Consumer duplicate rule matches?	Decision	W4P	YES: W4T; NO: W4N	Backend
W4T	Student-table grouping consumer?	Decision	W4D [YES]	YES: W4G; NO: W4B	Backend
W4G	Return canonical groups and duplicate IDs; UI hides repeated display rows	Process	W4T [YES]	next: W4E	Backend/frontend
W4B	Block scholar edit and show duplicate message	Input/Output	W4T [NO]	next: W4E	Frontend
W4N	Keep ordinary list display or allow edit to continue	Input/Output	W4D [NO]	next: W4E	Frontend
W4E	End duplicate check; no historical-row deletion	Start/End	W4G; W4B; W4N	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W4P	Weighted Record Linkage / Levenshtein	backend/grantor_algorithms.py:68-123	Pair result; consumer grouping adds score-only threshold
W4G	Canonical record preference	backend/grantor_algorithms.py:194-203	Prefer real account, nonarchived and complete records

W4 is a two-consumer comparison, not an account-approval workflow. ProviderDashboard handleSaveScholar calls findScholarDuplicate and returns on a match. AdminDashboard calls checkAdminStudentDuplicates and filters duplicate IDs from displayed studentProfiles. Neither behavior proves that every possible fuzzy match is blocked at official award creation.

Algorithm reference details (outside process nodes)

Mechanism	Compared fields and actual rule	Scope / limitations
Weighted pair linkage	Student ID .32; name .30; email .10; phone .08; course .08; year .04; city .04; province .04. Exact ID/email/phone; normalized Levenshtein for name/course/year/city/province; name=max(direct,token-sorted). Missing comparisons excluded and weights renormalized. Zero name is also omitted by name_similarity or None.	Birthdate, school, street/barangay and full address are not pair factors. Missing fields do not automatically mean mismatch.
Pair duplicate flag	Exact nonempty ID OR ((exact email OR exact phone) AND name>=.72) OR (name>=.82 AND weighted score>=.84). Reasons list fields>=.90.	Name similarities are computed before exact-ID flag; Levenshtein is not used only after ID failure. .90 is a reason threshold, not rejection.

Mechanism	Compared fields and actual rule	Scope / limitations
Student-table groups	Pair duplicate flag OR score >= configured threshold (default .84). Candidate joins first matching group; canonical selection favors real account/nonarchive/completeness.	This extra score-only branch can bypass pair name floor; not a global one-size threshold or confidence-category classifier.
Signup roster names	Roster exact-ID lookup first; name direct/sorted similarity>=.72 OR same last token and token overlap>=.50; missing name returns true.	Separate from weighted pair linkage. Exact official award still requires nonempty normalized name equality.
Browser ROG identity	ID weight .75; name .25; gate requires exact ID and, when both names exist, combined>=.85 plus name>=.70.	Caller logs inconclusive identity; not universal authoritative backend enforcement.
Roster preview/commit	If both IDs are present, compare exact IDs. Otherwise compare complete normalized name/course/year tuple; course aliases and comma-name normalization.	Different explicit IDs do not become a match just because profile text agrees. No-ID records may be retained unlinked; not automatically awarded.
Official assignment	Exact roster/student ID plus nonempty normalized name equality, current/unknown cycle handling, recognized program/provider, active account and no conflicting current award/commitment.	No Levenshtein confidence threshold determines this award. Transaction locks and rechecks are authoritative.

W5 - Existing roster comparison and authoritative commit

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W5S	Start existing manual or roster-entry workflow	Start/End	none	next: W5P	Grantor/admin
W5P	Authorize owner; validate cycle/program and compare proposed records	Process	W5S	next: W5D	Backend preview
W5D	Any invalid, ambiguous or conflicting row?	Decision	W5P	YES: W5B; NO: W5I	Backend
W5B	Return blocked preview with per-row reasons	Input/Output	W5D [YES]	next: W5E	API
W5I	Confirm ready batch	Input/Output	W5D [NO]	next: W5C	Grantor/admin
W5C	Commit RPC locks and rechecks current records and cycle	Process	W5I	pass: W5O; failure: W5X	Database
W5O	Return existing import, linked assignment or unlinked preserved row	Input/Output	W5C [pass]	next: W5E	API
W5X	Return conflict; transaction does not accept stale preview	Input/Output	W5C [failure]	next: W5E	API
W5E	End roster attempt	Start/End	W5B; W5O; W5X	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W5P	Exact ID / complete normalized profile key	backend/roster_workflow_service.py:141-146	Current-cycle cross-provider duplicate comparison
W5C	Transactional roster guard	supabase/migrations/20261026034945_cross_grantor_roster_identity_guard.sql	Authoritative commit, not merely frontend warning

W6 - Official award reconciliation and authorized resolution

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W6S	Start verified account reconciliation	Start/End	none	next: W6P	Backend
W6P	Find applicable active roster candidates and normalized current state	Process	W6S	next: W6N	Database
W6N	Applicable roster exists?	Decision	W6P	YES: W6D; NO: W6O	Database
W6D	One candidate and official assignment guards pass?	Decision	W6N [YES]	YES: W6A; NO: W6C	Database
W6A	Assign guarded official award or retain resolved existing award; synchronize affected state	Process	W6D [YES]; W6R [YES]	next: W6O	Database
W6C	Record unresolved conflict; restrict affected application/waitlist actions	Process	W6D [NO]	next: W6I	Database
W6I	Full administrator reviews conflict and supplies reasoned resolution	Input/Output	W6C	next: W6R	Full administrator
W6R	Authorized resolution and current guards pass?	Decision	W6I	YES: W6A; NO: W6X	Backend/database
W6X	Return error; preserve unresolved conflict	Input/Output	W6R [NO]	next: W6E	API
W6O	Return no assignment, assigned or unchanged state	Input/Output	W6N [NO]; W6A	next: W6E	API
W6E	End reconciliation/resolution	Start/End	W6X; W6O	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W6D	Exact official identity, cycle and commitment guards	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:69-194	No fuzzy automatic award
W6R	Protected conflict-resolution RPC	backend/roster_workflow_service.py:520-542	Candidate/existing/keep-current choices; reason required

Status, cycle and external-record handling

roster_record_active excludes archived, rejected, declined, withdrawn, inactive, cancelled/canceled, terminated and expired. Completed/finished are not in that exclusion list. roster_application_awarded includes locked, rosterConfirmedAward, requestedSOE and active/accepted/approved/awarded/finished/completed/finalized/protected statuses. Thus completed in the current or unknown cycle may still represent an award and block another assignment; do not write that every completed scholarship is automatically eligible elsewhere. Explicit previous-cycle records remain history and do not alone create a current-cycle award conflict. Unknown cycles are conservatively treated in the relevant blocker.

Pending applications are not automatically equivalent to official awards: competing pending applications may exist before commitment. A same-grantor open application and cooldown can block that grantor separately. Committing materials or assigning an authoritative award closes affected competing pending applications and releases slots. Withdrawn/rejected/inactive/cancelled status does not guarantee eligibility if another current award or unresolved conflict remains.

Full administrators may review and resolve recorded roster-assignment conflicts with an authorized choice and a reason (10-2000 characters in the service). A reviewer is not given a universal bypass. W6 is separate from suspected duplicate grouping W4. Account activation can succeed while roster conflict remains unresolved; account approval and scholarship award are not identical.

External/internal providers are represented by existing manually entered or uploaded roster records. No live external DOST/LGU/provider synchronization was established by the inspected paths. Unrecorded external scholarships cannot be detected by these comparisons. This is a coverage limitation, not evidence that the external award does not exist. Existing roster functionality is inspected only; no Excel-import modification is included.

W7 - Application and waitlist submission

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W7S	Start Apply action	Start/End	none	next: W7I	Student
W7I	Submit normalized scholarship, announcement and grantor identifiers	Input/Output	W7S	next: W7P	Frontend/API
W7P	Authorize owner and run current application, location and conflict guards	Process	W7I	next: W7D	Backend/database
W7D	Current eligibility permits application or waitlist?	Decision	W7P	YES: W7C; NO: W7X	Database
W7C	Application slot available?	Decision	W7D [YES]	YES: W7A; NO: W7W	Database
W7A	Reserve application and slot; attach profile snapshot where available	Process	W7C [YES]	next: W7O	Database/backend
W7W	Use existing waitlist policy and create eligible waitlist entry or return closure	Process	W7C [NO]	next: W7O	Database
W7O	Return application/waitlist result and refresh affected state	Input/Output	W7A; W7W	next: W7E	API
W7X	Return structured eligibility, access or conflict reason	Input/Output	W7D [NO]	next: W7E	API

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W7E	End application attempt	Start/End	W7O; W7X	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W7P	Cycle-aware commitment and scope guards	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:447-582	Same authoritative blocker also applies to waitlist eligibility; ranking is not approval

W8 - Stable Materials Request and transaction

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
W8S	Start Request Materials stage	Start/End	none	next: W8P	Student
W8P	Load and cache authoritative preflight for application	Process	W8S	next: W8D	Frontend/backend
W8D	Preflight eligible?	Decision	W8P	YES: W8I; NO: W8B	Frontend
W8I	Show stable confirmation; confirm request once	Input/Output	W8D [YES]	confirm: W8C; cancel: W8E	Student
W8C	Set Requesting; transaction locks and repeats shared readiness check	Process	W8I [confirm]	next: W8R	Frontend/database
W8R	Commit-time readiness still passes?	Decision	W8C	YES: W8A; NO: W8X	Database
W8A	Return existing request or create commitment and request; close competing pending applications	Process	W8R [YES]	next: W8O	Database
W8O	Return success; refresh materials, tracking, slots and inbox	Input/Output	W8A	next: W8E	Frontend/API
W8B	Keep request disabled; show blocker without repeated checking	Input/Output	W8D [NO]	next: W8E	Frontend
W8X	Return one structured conflict or service failure; no automatic request retry	Input/Output	W8R [NO]	next: W8E	API
W8E	End request attempt	Start/End	W8I [cancel]; W8O; W8B; W8X	end	System

Main node	Separate dashed reference box	Source function / block	Purpose
W8P	Normalized materials readiness	supabase/migrations/20261018090000_stable_materials_and_automatic_documents.sql:170 onward	Current COR/ROG or exemption, signup identity, approved profile revision and application/state requirements

Main node	Separate dashed reference box	Source function / block	Purpose
W8C	Idempotent locked transaction and shared preflight	supabase/migrations/20261012100000_repair_normalized_materials_workflow.sql:256 onward	Prevent stale client eligibility from authorizing conflicting commit

Flowchart boundaries: W1-W3 must not be merged into generic upload-then-admin-approval. W4 weighted suspected matches and W5 exact roster identity are different mechanisms. W6 official award guards are not a fuzzy confidence model. W7 recommendation/apply and W8 material commitment are different stages. Auxiliary per-action MPIN checks follow configured role/action policy and must not be redrawn as a replacement for User ID/password login or a universal mandatory condition.

SECTION 4 - Revision 3 Findings

Exact manuscript objectives

Specific Objective 1: To develop a centralized scholarship management system that enables students to apply and submit required documents, while allowing grantors and administrators to manage scholarship postings, evaluate applicants, and process approvals efficiently within a unified platform.

Specific Objective 2: To develop a system that validates student records, helps prevent multiple active scholarships, and supports duplication prevention using internal scholarship records and externally sourced scholarship records encoded into the system.

Specific Objective 3: To develop a system that provides scholarship recommendations, manages the scholarship lifecycle (application, approval, active status, renewal, and completion), and monitors student compliance and academic performance.

The wording supports the requested three system-development sections. Objectives 4-5 cover TAM and ISO/IEC 25010 evaluation: preserve their existing survey/evaluation content and organize them separately. Source code presence cannot establish user acceptance scores or software-quality effectiveness.

Objective	Role	Verified route / navigation	Capture action	Must be visible / demonstrated	Implementation
1	Grantor	/provider-dashboard/announcements	Open Create Announcement with a synthetic program; supply image, requirements and valid slots	Posting controls, visibility/application setting and resulting persisted announcement	backend/workflow_service.py create_grantor_announcement
1	Student	/student-dashboard/announcements/:source/:announcementId	Open an eligible synthetic announcement; complete Apply confirmation	Scholarship information and returned application/waitlist outcome	backend/scholarship_choice_service.py reserve_scholarship_application
1	Grantor / authorized admin	/provider-dashboard/applications; /admin requirements section	Open permitted student application; perform applicable review decision	Student/application context, stage, decision and resulting status; avoid real identifiers	backend/workflow_service.py update_admin_review
2	Prospective student	/signup	Use TEST-labelled COR/ROG; show one valid extraction and one correction	Document type, extracted academic period, applicable error; not actual personal records	backend/signup_service.py create_signup_document_batch
2	Student	/student-dashboard/profile	Upload synthetic replacement and view latest status; submit profile separately	Automatic COR/ROG status versus pending application profile	backend/student_profile_service.py upload_student_document / submit profile
2	Full administrator	/admin/students > Pending Accounts (internal tab)	Preview all applicable signup documents; observe single approval readiness	Email confirmation, preview indicators and account readiness; download alone must not count	backend/student_account_review_service.py
2	Authorized admin/reviewer	/admin/students > Document Review (internal tab)	Open newest synthetic Student Application Profile; approve/reject with reason	Profile-only queue, latest-version review and correction reason	backend/student_profile_service.py review_document_submission
2	Full administrator	/admin scholarship-program warning/conflict interface	Open a controlled current-cycle roster conflict and record authorized reasoned resolution	Both candidate assignments, cycle, restricted action and resolution result	backend/roster_workflow_service.py list/resolve_roster_conflict

Objective	Role	Verified route / navigation	Capture action	Must be visible / demonstrated	Implementation
3	Student	/student-dashboard/recommended-scholarships	Use known synthetic GWA and candidates; open recommendation cards	Recommendation labels, reasons and candidate details; do not relabel score as probability	backend/scholarship_rules.py recommend_scholarships
3	Student	/student-dashboard/scholarships	Open synthetic application at Request Materials; confirm once	Stage, stable confirmation, Requesting and successful material request state	backend/roster_workflow_service.py materials endpoints
3	Student / Full administrator	/student-dashboard/profile; /admin profile > Academic Cycle	In staging only, activate next cycle and restore prior cycle with controlled account	Renewal banner and per-cycle documents/tracking restored; capture each state	backend/academic_cycle_service.py; reversible-cycle SQL

Capture status for every entry: NOT VERIFIED in an authenticated running environment in this audit. Routes are source-derived, not proof that today's deployment has corresponding migrations, permissions or fixtures. Internal admin tabs are navigation instructions, not invented separate URLs. Use authorized synthetic/staging data; redact identifiers, email, contact details and document contents. Do not fabricate interface screenshots. Report capture date, role and fixture with the final figure.

O1 - Posting, application and evaluation overview

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
O1S	Start grantor posting	Start/End	none	next: O1I	Grantor
O1I	Enter announcement, image, requirements and application settings	Input/Output	O1S	next: O1D	Frontend/API
O1D	Owner, active grantor and posting fields valid?	Decision	O1I	YES: O1P; NO: O1X	Backend
O1P	Persist announcement; attempt/defer audience notification	Process	O1D [YES]	next: O1A	Backend/database
O1A	Student application or waitlist follows W7	Process	O1P	eligible application: O1R; blocked/waitlisted: O1E	Student/backend
O1R	Authorized evaluator reviews permitted application stage	Input/Output	O1A [eligible application]	next: O1V	Admin/grantor
O1V	Authorized current-stage decision valid?	Decision	O1R	YES: O1U; NO: O1X	Backend
O1U	Persist decision, release slot where applicable, update tracking/notifications	Process	O1V [YES]	next: O1E	Backend/database
O1X	Return validation/access/stage error	Input/Output	O1D [NO]; O1V [NO]	next: O1E	API
O1E	End posting/application/evaluation stage	Start/End	O1A [blocked/waitlisted]; O1U; O1X	end	System

Objective 1 implementation support: centralized persisted announcements, owner-scoped application reservation, existing waitlist policy and authorized stage decisions establish the implemented functions. A posted announcement, pending application, administrator decision, grantor confirmation and official award are not interchangeable outcomes. Notifications may fail/defer independently of the saved record. Do not claim all review writes form one atomic end-to-end transaction.

Objective 2 evidence interpretation

Use W1-W6 as a set of separately captioned process diagrams. Explain that parsing retrieves fields; validation checks only the rules implemented in that workflow; suspected duplicate linkage groups or blocks records according to its consumer; official roster assignment uses exact identity/cycle/commitment guards. This directly addresses Objective 2 without asserting comprehensive verification of every input or unrecorded external scholarship. Pair-match synthetic results are not official-conflict accuracy.

O3 - Recommendation, materials, signed SOE and cycle renewal overview

Node	Exact label	Symbol	Previous	Next / decision branches	Layer
O3S	Start student recommendation/lifecycle view	Start/End	none	next: O3R	Student
O3R	Load candidates; rank using R1; merge page priorities	Process	O3S	next: O3A	Frontend/backend
O3A	Submit separate application W7 and complete applicable review stages	Process	O3R	next: O3M	Student/admin/grantor
O3M	Request and approve/download materials through W8 and materials workflow	Process	O3A	next: O3I	Student/admin
O3I	Sign and upload authorized signed SOE original	Input/Output	O3M	next: O3D	Student
O3D	Owner, commitment, signing stage and file guards pass?	Decision	O3I	YES: O3F; NO: O3X	Backend/database
O3F	Store signed-SOE version; mark upload and finish; allow later admin reopening	Process	O3D [YES]	next: O3C	Database
O3C	Authorized academic-cycle transition requested?	Decision	O3F	YES: O3V; NO: O3E	Root/full admin
O3V	Snapshot departing cycle; restore target snapshot or reconstruct/start renewal	Process	O3C [YES]	success: O3O; stale/error: O3X	Database
O3O	Show current-cycle documents, tracking and renewal banner when incomplete	Input/Output	O3V [success]	next: O3E	Student frontend
O3X	Return current operation error; do not claim transition or finish	Input/Output	O3D [NO]; O3V [stale/error]	next: O3E	API
O3E	End lifecycle operation	Start/End	O3C [NO]; O3O; O3X	end	System

O3 links independently triggered subprocesses and is not one automatic transaction. Standard and KWSP tracking have different stage sequences. Materials creation is not materials approval. Signed SOE finalization itself adds finish after its guards; administrator reopening is a subsequent event. Reversible cycle transitions preserve exact post-deployment snapshots where available and use best-effort reconstruction for older cycles. Active identity is not renewed each semester.

Authentic code excerpts and objective explanations

E1 - Application reserve delegates authoritative slot/waitlist transaction

Objective 1; exact source backend/scholarship_choice_service.py:96-110. No omitted lines within this excerpt.

```
    if not student_id or not announcement_id or not grantor_id:
        return {"ok": False, "reason": "missing_application_identity"}
    # Both identifiers are server-generated. Browser retries match the stored
    # student/grantor/offering tuple instead of trusting a caller's record ID.
    application_id = str(uuid4())
    application_number = f"{student_id[-3:]}-{uuid4().hex[:8]}"
    # The transaction checks the normalized award and its cycle. A cached or
    # historical commitment in a hydrated response must not decide eligibility.
    result = supabase_rpc("reserve_or_waitlist_scholarship", {
        "p_student_id": student_id, "p_announcement_id": announcement_id,
        "p_grantor_id": grantor_id, "p_application_id": application_id,
        "p_application_number": application_number,
    })
    if not result.get("ok"):
        reason = result.get("reason") or "slot_reservation_failed"
```

Input is authorized student and normalized program identifiers; server-generated application identity is passed to the reservation RPC. Output is normalized success/blocker data. This supports centralized submission, not automatic award.

E2 - Signup COR owner and academic-period validation

Objective 2; exact source backend/signup_service.py:203-220. No omitted lines within this excerpt.

```
    if cor_scan.get("isValidCorDocument") is False:
        raise HTTPException(status_code=422, detail="invalid_cor_document_title")
    scanned_student_id = normalize_student_id(cor_scan.get("studentId"))
    if not scanned_student_id or scanned_student_id != normalized_student_id:
        raise HTTPException(status_code=422, detail="cor_student_id_mismatch")
    detected_cor_year = _signup_year(cor_scan.get("year"))
    submitted_year = _signup_year(declared_year)
    if declared_year and not submitted_year:
        raise HTTPException(status_code=422, detail="invalid_year_level")
    if not detected_cor_year and not submitted_year:
        raise HTTPException(status_code=422, detail="cor_year_level_not_detected")
    year = submitted_year or detected_cor_year
    year_level_mismatch = bool(submitted_year and detected_cor_year and submitted_year != detected_cor_year)

    current_cycle = get_current_semester_tag()
    cor_cycle = build_semester_tag(cor_scan)
    if not cor_cycle or cor_cycle != current_cycle:
        raise HTTPException(status_code=422, detail="cor_cycle_mismatch")
```

Receives parsed COR and declared signup ID/year. It rejects a wrong COR ID and cycle, while deriving year/review warning. It does not establish that the ROG owner or all fields were validated.

E3 - Weighted pair decision

Objective 2; exact source backend/grantor_algorithms.py:92-110. No omitted lines within this excerpt.

```
  fields = [
      {"label": "student ID", "weight": 0.32, "value": comparable_similarity(candidate_student_id, existing_student_id, True)},
      {"label": "name", "weight": 0.30, "value": name_similarity or None},
      {"label": "email", "weight": 0.10, "value": comparable_similarity(candidate_email, existing_email, True)},
      {"label": "contact number", "weight": 0.08, "value": comparable_similarity(candidate_phone, existing_phone, True)},
      {"label": "course", "weight": 0.08, "value": comparable_similarity(candidate.get("course"), existing.get("course"))},
      {"label": "year level", "weight": 0.04, "value": comparable_similarity(candidate.get("yearLevel") or candidate.get("year"),
  existing.get("yearLevel") or existing.get("year"))},
      {"label": "city", "weight": 0.04, "value": comparable_similarity(candidate.get("city"), existing.get("city"))},
      {"label": "province", "weight": 0.04, "value": comparable_similarity(candidate.get("province"), existing.get("province"))},
  ]
  compared_fields = [field for field in fields if field["value"] is not None]
  compared_weight = sum(field["weight"] for field in compared_fields)
  weighted_score = (
      sum(float(field["value"]) * field["weight"] for field in compared_fields) / compared_weight
      if compared_weight > 0
      else 0
  )
  strong_identifier_match = exact_student_id or ((exact_email or exact_phone) and name_similarity >= 0.72)
  is_duplicate = strong_identifier_match or (name_similarity >= 0.82 and weighted_score >= 0.84)
```

Receives candidate and existing record; normalizes weighted available fields and applies exact-identifier or similarity branches. The consumer determines grouping versus blocking.

E4 - Official award identity and cycle checks

Objective 2; exact source supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:81-104. No omitted lines within this excerpt.

```
  select data into student_data from public.students where id = p_student_id for update;
  if not found then raise exception 'student_not_found'; end if;
  if not public.roster_record_active(student_data) or student_data->>'disabled' = 'true'
    or student_data->>'adminBlocked' = 'true' or student_data->>'isPending' = 'true'
    or lower(coalesce(student_data->>'status', '')) in ('blocked', 'disabled', 'pending') then
    raise exception 'student_account_blocked';
  end if;
  select data into roster_data from public.grantor_portal_scholars where id = p_roster_id and parent_id = p_grantor_id for update;
  if not found or coalesce(roster_data->>'studentId', roster_data->>'studentnumber', roster_data->>'studentNumber', '') <> p_student_id
    or not public.roster_record_active(roster_data) then raise exception 'roster_match_not_found'; end if;
  -- Untagged legacy rows require staff verification; never guess their semester.
  if public.roster_cycle(roster_data) <> cycle then raise exception 'roster_cycle_requires_verification'; end if;
  if public.roster_name_key(coalesce(roster_data->>'fullName', concat_ws(' ', roster_data->>'fname', roster_data->>'mname', roster_data->>'lname'))) =
    ''
    or public.roster_name_key(coalesce(roster_data->>'fullName', concat_ws(' ', roster_data->>'fname', roster_data->>'mname', roster_data->>'lname')))
      <> public.roster_name_key(coalesce(student_data->>'fullName', concat_ws(' ', student_data->>'fname', student_data->>'mname',
    student_data->>'lname'))) then
    raise exception 'roster_identity_mismatch';
  end if;
  target_scholarship_id := coalesce(nullif(roster_data->>'scholarshipId', ''), roster_data->>'announcementId');
  scholarship_name := coalesce(roster_data->>'scholarshipTitle', roster_data->>'scholarshipName', 'Scholarship');
  if exists(select 1 from public.grantor_portal_scholars where id <> p_roster_id
    and coalesce(data->>'studentId', data->>'studentnumber', data->>'studentNumber') = p_student_id
    and public.roster_record_active(data) and public.roster_cycle(data) in ('', cycle)) then
    raise exception 'scholarship_already_committed';
  end if;
```

Receives locked student/roster data in the assignment transaction. Active identity, normalized nonempty name and applicable cycle must agree, and competing current rosters restrict assignment.

E5 - Active academic replacement policy

Objective 2; exact source backend/student_profile_service.py:900-917. No omitted lines within this excerpt.

```
if document_type in {"cor", "rog"}:
    if content_type != "application/pdf":
        raise HTTPException(status_code=415, detail="pdf_required")
    scan = parse_pdf_document(body, "cor" if document_type == "cor" else "cog")
    if document_type == "cor":
        detected = str(scan.get("documentTitle") or "")
        kind = "advising_slip" if detected == "Advising Slip" else "cor"
        mode = requirements["corMode"]
        if not scan.get("isValidCorDocument") or (mode == "cor_only" and kind != "cor") or (mode == "advising_only" and kind != "advising_slip"):
            exception = _data_rows("student_document_exceptions", {
                "data->>studentId": student_id, "data->>academicCycle": cycle,
                "data->>exceptionType": "advising_slip", "data->>status": "active",
            }, limit=1)
            if not (kind == "advising_slip" and exception):
                raise HTTPException(status_code=422, detail="cor_policy_not_satisfied")
        document_kind = kind
    elif not scan.get("isValidCogDocument"):
        raise HTTPException(status_code=422, detail="invalid_rog_document")
```

Receives authenticated student PDF and parser output; enforces document type/title/COR policy, including exception lookup. This excerpt does not contain signup-level owner/period validation.

E6 - Recommendation hard exclusions

Objective 3; exact source backend/scholarship_rules.py:146-157. No omitted lines within this excerpt.

```
for item in scholarships:
    if item.get("applicationsBlocked") is True:
        continue
    if item.get("applicationEnabled") is False and item.get("applyOpen") is not True:
        continue

    minimum_gwa = _to_float(item.get("minimumGwa") or item.get("minGwa") or item.get("minimumGrade"))
    provider = item.get("providerLabel") or item.get("grantorName") or item.get("title") or item.get("name") or ""
    if minimum_gwa is None:
        minimum_gwa = _to_float(get_scholarship_policy(provider)["minGwa"]) or 2.25
    if student_gwa is None or student_gwa > minimum_gwa:
        continue
```

Receives each candidate after GWA preparation; excludes explicit blocked, disabled-without-override and academically ineligible candidates. Remaining items proceed to scoring.

E7 - Grade and popularity factors

Objective 3; exact source backend/scholarship_rules.py:159-171. No omitted lines within this excerpt.

```
        reasons = []
        grade_margin = max(0.0, minimum_gwa - student_gwa)
        grade_score = min(40.0, 24.0 + (grade_margin * 12.0))
        if grade_margin >= 0.5:
            reasons.append("Strong GWA match")
        else:
            reasons.append("Meets minimum GWA")

        roster_count = _to_int(item.get("rosterCount"))
        popularity_weight = 30.0 if is_high_grade_student else 22.0
        popularity_score = (roster_count / max_roster * popularity_weight) if max_roster > 0 else 0.0
        if roster_count > 0:
            reasons.append(f"{roster_count} scholar roster")
```

Computes grade fit and relative roster popularity after filtering. These are deterministic ranking factors, not learned probabilities.

E8 - Recommendation output and ranking

Objective 3; exact source backend/scholarship_rules.py:185-205. No omitted lines within this excerpt.

```
        score = grade_score + popularity_score + location_score + completeness_score
        label = _recommendation_label(score, reasons)
        recommendations.append(
            {
                "item": item,
                "score": round(score, 4),
                "label": label,
                "reasons": reasons[:4],
                "criteria": {
                    "gwa": student_gwa,
                    "minimumGwa": minimum_gwa,
                    "rosterCount": roster_count,
                    "gradeScore": round(grade_score, 2),
                    "popularityScore": round(popularity_score, 2),
                    "locationScore": round(location_score, 2),
                },
            }
        )

    recommendations.sort(key=lambda row: row["score"], reverse=True)
    return {
```

Appends rounded scores, labels and criteria then sorts descending. Page priority merging occurs later and may alter the visible order.

E9 - Signed SOE completes workflow

Objective 3; exact source supabase/migrations/20260924125213_security_history_signed_soe.sql:311-325. No omitted lines within this excerpt.

```
  completed := coalesce(tracking->'completedStepIds', '[]'::jsonb);
  if not (completed ? 'signed_soe_upload') then completed := completed || '"signed_soe_upload"'::jsonb; end if;
  if not (completed ? 'finish') then completed := completed || '"finish"'::jsonb; end if;
  tracking := tracking || jsonb_build_object('completedStepIds', completed, 'currentStepId', 'finish', 'updatedAt', now_text,
    'history', coalesce(tracking->'history', '[]'::jsonb) || jsonb_build_array(
      jsonb_build_object('stepId','signed_soe_upload','label','Upload Signed
```

```
  SOE','completedAt',now_text,'completedBy','student','completionSource','signed_soe_submission'),

  jsonb_build_object('stepId','finish','label','Finish','completedAt',now_text,'completedBy','system','completionSource','signed_soe_submission')));
update public.scholarship_applications set data = d || jsonb_build_object('tracking', tracking,
  'signedSoeSubmissionId', p_submission_id, 'signedSoeStatus', 'submitted', 'finishedAt', now_text, 'updatedAt', now_text), updated_at = now() where
  id = p_application_id;
insert into public.signed_soe_audit_events(id, submission_id, application_id, event_type, data)
  values('signed_soe_submitted_' || p_submission_id, p_submission_id, p_application_id, 'submitted',
  jsonb_build_object('studentId',p_student_id,'cycle',p_cycle,'createdAt',now_text));
insert into public.student_history_events(id, student_id, event_type, academic_cycle, occurred_at, related_type, related_id, route, description,
  safe_data)
  values('history_signed_soe_' || p_submission_id, p_student_id, 'signed_soe_submitted', p_cycle, now(), 'application', p_application_id,
    '/student-dashboard/scholarships', 'Signed SOE submitted. This scholarship cycle is finished.',
  jsonb_build_object('submissionId',p_submission_id)) on conflict(id) do nothing;
return jsonb_build_object('submissionId', p_submission_id, 'version', next_version, 'status', 'submitted', 'finishCompleted', true);
```

The transaction adds signed_soe_upload and finish, updates the application and writes history/audit. It demonstrates why a mandatory pre-finish administrator-approval claim must be corrected.

Exact manuscript editing instructions

Do not replace the entire chapter. Preserve existing valid module descriptions and figures, move them under the matching objectives, and correct only contradicted wording. New figure names below use provisional 4.X-O1 /4.X-O2 /4.X-O3 tokens; final numbering and cross-references must be updated in Word after moving figures. Current printed pages are location aids, not final pagination.

Chapter IV opening

WHAT TO FIND? The chapter is divided into three major sections. [Matched direct-body paragraphs: [625]]

WHAT TO DO? Replace only this organizational paragraph (direct-body paragraph625; printed125-126). Preserve the preliminary-study discussion as context.

WHAT TO PUT? This chapter presents the preliminary findings and the system results according to the specific objectives. The three system-development objectives are presented using actual interface screenshots, verified process flowcharts and source-code excerpts. The Technology Acceptance Model and ISO/IEC 25010 evaluation results are discussed separately under their corresponding evaluation objectives.

Objective 1 organization

WHAT TO FIND? II. Designing and Developing a Centralized Scholarship Management and Tracking System [Matched direct-body paragraphs: [632]]

WHAT TO DO? Use this exact heading as the start of the objective-based development portion (paragraph632). Preserve its valid introductory module text. Add the Objective1 subsection; move Figures4.1-4.4,4.6,4.8-4.10 and relevant approval Figure4.12 beside its discussion.

WHAT TO PUT? Specific Objective 1: Centralized Scholarship Posting, Application and Evaluation. Figure4.X-O1a. Centralized scholarship posting and application interfaces. Figure4.X-O1a presents the grantor announcement controls and the student submission interface. The system stores scholarship information and processes owner-scoped applications or waitlist outcomes. Figure4.X-O1b. Scholarship posting, application and evaluation process. Use O1 and its W7 subprocess. The process validates authorization and posting data before persistence, then evaluates application eligibility and permitted stage decisions. Figure4.X-O1c. Application reservation implementation.
code snippet

Use excerpt E1. The source passes authorized student and scholarship identifiers to the reservation transaction; the result distinguishes a submitted application from a waitlist or blocked attempt.

Objective 2 organization

WHAT TO FIND? Figure 4.11. Duplicate Scholarship Validation [Matched direct-body paragraphs: [677]]

WHAT TO DO? Move the figure and its valid explanation (paragraph677; printed140-141) under a new Objective2 subsection. Add actual registration/profile/conflict screenshots from the capture matrix. Preserve test tables after implementation evidence; distinguish parser gaps.

WHAT TO PUT? Specific Objective 2: Student-Record Validation and Scholarship Duplication Prevention. Figure4.X-O2a. Student document validation and recorded conflict review. Figure4.X-O2a shows document-specific validation outcomes and authorized review of recorded scholarship conflicts. Extracting a field does not by itself verify that field. Figure4.X-O2b. Separate validation and conflict-prevention processes. Use W1-W6 as separately captioned diagrams, with dashed references to the mechanisms listed in Section3. Suspected duplicate grouping, exact roster identity checks and transactional award prevention perform distinct functions. Figure4.X-O2c. Record matching and official award guards.
code snippet
Use excerpts E2-E4. The code demonstrates signup COR checks, weighted suspected-record matching and exact cycle-aware award guards. Existing imported records are compared; external awards not recorded in the system remain outside detection coverage.

Objective 3 organization

WHAT TO FIND? Figure 4.5. Scholarship Recommendation [Matched direct-body paragraphs: [659]]

WHAT TO DO? Move recommendation Figure4.5 (paragraph659), tracking Figure4.7 and monitoring/renewal/notification Figures4.13-4.15 into Objective3; retain relevant report Figure4.16 where the monitoring discussion refers to it.

WHAT TO PUT? Specific Objective 3: Scholarship Recommendations and Lifecycle Monitoring. Figure4.X-O3a. Recommendation and application-tracking interfaces. Figure4.X-O3a presents ranked scholarship options and the current application stage. A recommendation is guidance and does not constitute eligibility approval or an award. Figure4.X-O3b. Recommendation and lifecycle process. Use R1 and O3 as separate diagrams; R1 describes deterministic recommendation filtering and scoring, while O3 references application, materials, signed-SOE and cycle processes. Figure4.X-O3c. Recommendation scoring and lifecycle completion implementation.
code snippet
Use E6-E9. Candidates passing explicit open-state and GWA checks receive grade-fit, roster-popularity, location and candidate-image scores. The page may prioritize invitations and administrator notices above numerical ranking. After authorized signed-SOE submission, the finalizer marks completion; administrators may subsequently reopen the submission.

Recommendation terminology

WHAT TO FIND? The system uses a rule-based Decision Tree algorithm based on predefined eligibility filtering and deterministic weighted recommendation scoring. [Matched direct-body paragraphs: [636]]

WHAT TO DO? Replace this sentence within the Scholarship Recommendation Module paragraph636 (printed127). Search other Decision Tree implementation claims individually; preserve correct ChapterIII formulas.

WHAT TO PUT? The system uses Weighted Recommendation Scoring, consisting of rule-based candidate filtering followed by deterministic scoring of grade fit, roster popularity, location information and candidate-image presence. The decision-flowchart represents implemented branches rather than a trained machine-learning Decision Tree. Recommendation results do not automatically approve applications.

Evaluation organization

WHAT TO FIND? Results and Discussion of the System's Evaluation [Matched direct-body paragraphs: [704]]

WHAT TO DO? Preserve this heading and all verified evaluation results (paragraph704 onward). Place COR/ROG functional evidence under Objective2 or explicitly cross-reference it; retain its limitations. Organize TAM and ISO results as Objectives4 and5 without inventing data.

WHAT TO PUT? The following functional-test evidence evaluates the stated test cases only. It is separate from user-acceptance and software-quality evaluation. Document parser results and synthetic matching metrics are not estimates of overall student-record accuracy, external-scholarship coverage or production security.

Signed-SOE clarification

WHAT TO FIND? The signed SOE is uploaded and verified before the scholarship cycle can be marked as completed. [Matched direct-body paragraphs: [592]]

WHAT TO DO? Clarify only this sentence in the Figure3.8 explanation (direct-body paragraph592; printed120-121). The word verified is ambiguous: the text does not explicitly state that a separate administrator must approve the upload. Preserve the administrator material-request review described earlier in the paragraph. Use finalize_signed_soe_submission as authority.

WHAT TO PUT? For eligible committed applications, a signed-SOE upload is validated for ownership, file constraints and the required signing stage. Successful finalization stores a versioned submission and marks the upload and finish steps complete. An administrator may later reopen the submission for correction; prior administrator approval is not a mandatory finish condition in this implementation.

Objective	Implemented feature	Screenshot location	Process reference	Code source	Verification status
1	Posting, submission/waitlist, stage evaluation	Capture matrix rows1-3	O1;W7	E1; workflow_service creation/review	IMPLEMENTED; authenticated capture and deployment NOT VERIFIED
2	Document-specific validation; linkage; exact official conflicts	Capture matrix rows4-8	W1-W6	E2-E5;G;C; latest roster commit	IMPLEMENTED/PARTIAL; recorded synthetic tests and negative gaps; no live external feed
3	Deterministic recommendation, materials, monitoring and reversible cycles	Capture matrix rows9-11	R1;W8;O3	E6-E9; materials and cycle SQL	IMPLEMENTED; production restoration and actual screenshot evidence NOT VERIFIED

SECTION 5 - Source Code Evidence Index

Revision	Feature	File path	Function or class	Line range	Verified behavior
1	Core recommendation	backend/scholarship_rules.py	recommend_scholarships	137-216	Exclusion filters, factor scores, label, descending rank; no automatic award
1	Location factor	backend/scholarship_rules.py	_location_score	239-262	Flat province/city/barangay bonuses, not geographic distance or scope enforcement
1	Label precedence	backend/scholarship_rules.py	_recommendation_label	265-274	78-point label first; then nearest, roster, strong GWA, available
1	Authorized recommendation API	backend/main.py	recommend_scholarships_endpoint	797-803	Stored authenticated student replaces caller student; candidates remain caller supplied
1	Candidate loader and degraded fallback	src/services/recommendedScholarshipService.js	source block	76-303	Open announcement candidates; cached roster counts; selected network failures use local scoring
1	Displayed priority	src/pages/StudentRecommendedScholarshipsPage.jsx	source block	150-224	Invitations and notices merged; duplicate scholarships removed; priority then score
1,2	Application commitment/cooldown	src/services/scholarshipChoiceService.js	source block	1-84	Frontend commitment and same-grantor checks are separate from core ranking
2,3	PDF extraction	backend/document_scanner.py	parse_pdf_document	926-970	PDF text and positioned year; ROG table/word/text Final-column fallback
2,3	Signup file batch	backend/signup_service.py	create_signup_document_batch	172-306	COR ID/cycle/title/policy; ROG title/previous period/academic concerns; no ROG owner-ID equality here
2,3	Final signup	backend/signup_service.py	validate_student_signup	633-820	Required profile/contact data, batch, cycle/year checks, availability and roster identity conflicts
2,3	Signup persistence	backend/signup_service.py	finalize_student_signup	896-1055	Pending signup and versioned document records; not an award
2,3	Signup roster names	backend/signup_service.py	is_similar_roster_name	549-562	Direct/sorted name similarity or last-token overlap after roster-ID lookup
2,3	Signup roster lookup	backend/signup_service.py	find_roster_identity_conflicts	575-612	Existing roster ID aliases then name comparison; not universal fuzzy-first matching
2,3	Profile replacement	backend/student_profile_service.py	upload_student_document	885-967	PDF parser/title/policy; automatic acceptance after system review; fewer cross-document checks than signup

Revision	Feature	File path	Function or class	Line range	Verified behavior
2,3	Profile PDF submission	backend/student_profile_service.py	submit_student_profile	846-882	Profile validation, signature, PDF revision and pending administrator review
2,3	Administrator review	backend/student_profile_service.py	review_document_submission	999-1063	Latest profile and pending state; rejection reason; review record and summary update
2,3	Weighted duplicate pair	backend/grantor_algorithms.py	evaluate_scholar_duplicate	68-123	Exact identifiers plus normalized Levenshtein fields; duplicate Boolean and score
2,3	Duplicate groups	backend/grantor_algorithms.py	check_student_table_duplicates	206-262	Pair flag OR configured score threshold; canonical record and duplicate IDs
2,3	Roster preview	backend/roster_workflow_service.py	preview_roster_import	258-465	Current cycle, provider/program, exact ID or complete normalized profile and conflict outcomes
2,3	Roster commit	backend/roster_workflow_service.py	commit_roster_import	468-480	Delegates authoritative transaction; preview is not final authority
2,3	Conflict resolution	backend/roster_workflow_service.py	resolve_roster_conflict	520-542	Full administrator, reason, explicit resolution via RPC
2,3	Materials preflight	backend/roster_workflow_service.py	preflight_student_materials	545-560	Authorized application ID to shared SQL preflight
2,3	Materials request	backend/roster_workflow_service.py	request_student_materials	563-600	Transactional request, structured conflict response
2,3	Application reserve	backend/scholarship_choice_service.py	reserve_scholarship_application	72-113	Student owner identity and normalized IDs; reserve-or-waitlist RPC
2,3	Announcement creation	backend/workflow_service.py	create_grantor_announcement	1383-1468	Owned active grantor; image and application-slot validation; persisted announcement
2,3	Application review	backend/workflow_service.py	update_admin_review	734-897	Authorized updates and immutable identity restrictions; stage-specific decisions
2,3	Signed SOE	backend/security_history_service.py	upload_signed_soe	110-150	Private original and optional processing copy; authorized finalizer
2,3	Cycle transition	backend/academic_cycle_service.py	activate_academic_cycle	41-97	Cycle format and expected-current-cycle; reversible SQL transition
2,3	Cycle-aware award guards	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql	source block	26-42	Active roster exclusions and awarded application statuses; completed remains an award
2,3	Official roster assignment	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql	source block	69-194	Exact identity, cycle, competing awards, locks, idempotency and application/state writes
2,3	Reconciliation	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql	source block	196-254	Verified account attempts assignment; multiple/error states record conflicts

Revision	Feature	File path	Function or class	Line range	Verified behavior
2,3	Application blocker	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql	source block	447-582	Current/unknown-cycle conflicts and commitment gate application/waitlist
2,3	Latest roster commit	supabase/migrations/20261026034945_cross_grantor_roster_identity_guard.sql	source block	1-40	Exact ID when both available, otherwise complete normalized name/course/year; later commit definition
2,3	Latest profile identity key	supabase/migrations/20261026043502_manual_roster_name_and_course_matching.sql	source block	1-32	Course aliases, comma name normalization and year key
2,3	Normalized readiness	supabase/migrations/20261018090000_stable_materials_and_automatic_documents.sql	source block	170-205	Current-cycle normalized document readiness; inspect full function for all blockers
2,3	Materials transaction	supabase/migrations/20261012100000_repair_normalized_materials_workflow.sql	source block	256-330	Locks, idempotency, same preflight, commitment and competing application closure
2,3	Signed SOE finish	supabase/migrations/20260924125213_security_history_signed_soe.sql	source block	291-326	Finalizer marks signed upload and finish together; admin may reopen separately
2,3	Reversible cycle	supabase/migrations/20261011135631_reversible_academic_cycle_management.sql	source block	145-175	Transition snapshots; restore stored state or reconstruct legacy cycle

Source locations describe the checked-out revision as of inspection. Later migrations can replace earlier functions: roster identity keys use 20261026043502; roster commit uses 20261026034945 rather than the older Oct23 commit body. The Oct23 reconciliation and award guard remain separate functions. Source_Manifest.json records hashes for reproducibility; rerun verification if source changes.

SECTION 6 - Thesis and Implementation Discrepancies

Claim/problem	Evidence-based difference	Revision action	Status
Recommendation described as Decision Tree	Existing weighted-scoring explanations coexist with Decision Tree implementation claims (printed18,98,105,122,127).	Use Weighted Recommendation Scoring; retain a rule-based branch flowchart, not a learned model.	IMPLEMENTED, terminology correction required
Avoid universal fields/ID-first fuzzy sequence	The current manuscript already lists the eight weighted fields correctly in its record-linkage discussion. Pair linkage computes name before identifier flags; signup and official roster use different rules.	Preserve correct existing fields/weights; qualify any generic ID-first or fallback wording by workflow rather than replacing accurate text.	IMPLEMENTED; clarify scope, not a wholesale correction
One duplicate threshold	Pair linkage .72/.82/.84; table grouping adds configurable default .84; signup name .72/overlap.50; official award exact.	Explain each gate and consumer; .90 is a reasons threshold only.	IMPLEMENTED
All documents undergo admin review	Active COR/ROG/identity replacements use system review; newest application profiles use manual review.	Separate signup account verification, active academic replacement and profile approval.	IMPLEMENTED/PARTIAL cross-field rules
Comprehensive academic verification	Four stored negative probes show missing owner/period/required-field enforcement in distinct paths.	State exact checks and limitations; do not generalize passing fixture extraction to correctness.	RECORDED negative evidence
Manual-only account activation figure	Manual enabled waits for admin; disabled may activate after confirmed email and approved required signup docs.	Add automatic and already-active branches; mail delivery separate.	IMPLEMENTED; deployment NOT VERIFIED
Completed always allows another scholarship	Current/unknown-cycle completed/finished may remain awarded; earlier explicit cycles differ.	Describe actual status/cycle predicates, not unconditional allow/block.	IMPLEMENTED
External records fully available	Only recorded imported/manual provider data is compared; no live provider synchronization established.	State coverage boundary and unlinked no-ID handling; do not claim unseen external awards are detected.	PARTIAL coverage
Ambiguous verification before signed-SOE finish	Paragraph592 says uploaded and verified, but does not explicitly identify a separate administrator approval. SQL marks upload and finish on eligible submission, with later reopening.	Clarify verification as the implemented ownership/stage/file guards; do not assert an explicit manuscript contradiction that the sentence does not contain.	IMPLEMENTED; wording clarification
ChapterIV organized by interfaces rather than objectives	Current three broad sections and sequential Figures4.1-4.16 do not consistently follow screenshot/process/code per objective.	Reorganize with exact anchors above; preserve TAM/ISO data and current valid figures.	MANUSCRIPT organization revision
Accuracy or security success from code presence	Local fixtures, mocked boundaries and isolated SQL tests cannot establish real-population accuracy, production authorization or security certification.	Label evidence source, sample size, execution date and outstanding environment checks.	NOT VERIFIED for production/general accuracy

SECTION 7 - Unresolved Questions

Genuine evidence gap	What is needed	Why source alone cannot answer
Production deployment and active migrations	Read-only deployed version/migration inventory plus authorized staging/live smoke-test evidence. No credentials in the report.	Presence of migrations does not prove execution or provider configuration.
Authenticated screenshots and actual responsive state	Authorized synthetic account access; dated captures following the matrix, with role/fixture and redactions.	No screenshots were fabricated or captured during this audit.
General student-match and conflict accuracy	Independently labeled representative pairs and cycle/status conflict cases; clear denominators and false-positive/negative counts.	Ten synthetic pairs do not estimate university-wide accuracy.
OCR runtime and image/scanned-document correctness	Working Poppler/Tesseract configuration plus real-parser OCR tests using authorized representative scans and transcribed ground truth.	Recorded OCR probes were blocked by missing binaries; health flag alone is insufficient.
Live restoration, idempotency and concurrency	Controlled staging 1ST->2ND->1ST->2ND records and concurrent transaction test logs.	Source shows guards/snapshots, not an observed deployed restoration or race result.
Final figure numbering and pagination	Researcher finalizes Word reorganization then updates captions, cross-references and generated PDF.	Provisional new figure tokens deliberately avoid inventing final page/figure numbers.

No question is left open where current source already establishes the rule. Outstanding evidence does not authorize production-changing actions. No source/UI/database/configuration changes or Revision4 implementation were performed. The manuscript remains untouched; use the edit guide to make controlled revisions manually.